

Valid Paranthesis String

Problem Statement: Find the validity of an input string *s* that only contains the letters '(', ')' and '*'. A string entered is legitimate if

- Any left parenthesis '(' must have a corresponding right parenthesis ')'.
• right parenthesis ')' must have a corresponding left parenthesis '('.
- Left parenthesis '(' must go before the corresponding right parenthesis ')'.
• could be treated as a single right parenthesis ')' or a single left parenthesis '(' or an empty string "".

Examples

Input : *s* = (*)

Output : True

Explanation : The * can be replaced by an opening '(' bracket. The string after replacing the * mark is "()" and is a valid string.

Input : *s* = *()

Output : false

Explanation : The * replaced with any bracket does not form a valid string.

Brute Force Approach

Algorithm

We are given a string that can contain three characters:

- '(' which represents an opening bracket
- ')' which represents a closing bracket
- '*' which can represent an opening bracket, a closing bracket, or be an empty string

The idea is to explore every possible way to interpret the '*' characters. At each step, we try all interpretations for '*', and check whether the resulting string is a valid parentheses string. This approach uses recursion (backtracking) to explore all combinations.

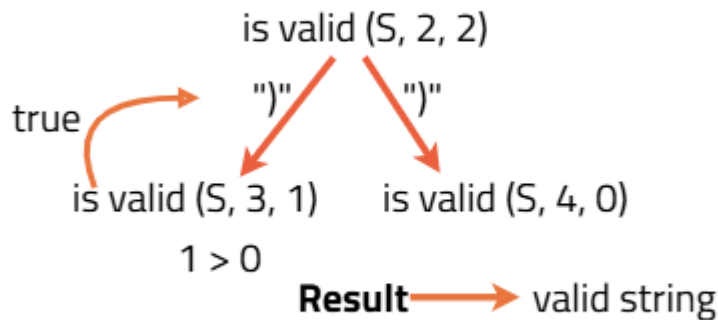
1. Define a recursive function that takes the current index and the number of open brackets.
2. If the number of open brackets becomes negative at any point, return false (invalid string).
3. If we reach the end of the string:
 - If the number of open brackets is zero, return true (valid string).
 - Otherwise, return false.
4. If the current character is '(', increase the open bracket count and call the function recursively.
5. If the character is ')', decrease the open bracket count and call the function recursively.
6. If the character is '*', try all three possibilities:
 - Ignore it: call the function without changing the open count.
 - Treat it as '(': increase open count and call recursively.
 - Treat it as ')': decrease open count and call recursively.
7. If any of the recursive calls return true, return true.
8. If none return true, return false.

S = (*)
is valid (S, 0, 0)

"(" → is valid (S, 1, 1)

"))" → is valid (S, 2, 0)
"(" → is valid (S, 2, 1)
"(" → is valid (S, 2, 2)

"))" → is valid (S, 3, 0)
"))" → is valid (S, 4, -1)
-1 < 0
false



Code

```

import java.util.*;
public class Solution {

    // Recursive function to check if the parenthesis string is valid
    public boolean isValid(String s, int index, int open) {

        // If the number of open parentheses becomes negative, return false
        if (open < 0) return false;

        // If we've reached the end of the string, return true if all open
        // brackets are closed
        if (index == s.length()) return open == 0;

        // Get the current character
        char c = s.charAt(index);

        // If it's an opening bracket '(', increase the count of open
        if (c == '(') {
            return isValid(s, index + 1, open + 1);
        }

        // If it's a closing bracket ')', decrease the count of open
        else if (c == ')') {
            return isValid(s, index + 1, open - 1);
        }

        // If it's '*', we try all 3 possibilities:
        // 1. Treat '*' as empty string
        // 2. Treat '*' as '('
        // 3. Treat '*' as ')'
        else {
            return isValid(s, index + 1, open) ||
                isValid(s, index + 1, open + 1) ||
                isValid(s, index + 1, open - 1);
        }
    }
}

// Driver class
class Main {
    public static void main(String[] args) {

```

```

    java.util.Scanner sc = new java.util.Scanner(System.in);

    // Prompt the user to enter the string
    System.out.print("Enter the string: ");
    String input = sc.nextLine();

    // Create Solution object and call the recursive function
    Solution sol = new Solution();
    if (sol.isValid(input, 0, 0)) {
        System.out.println("Valid parenthesis string");
    } else {
        System.out.println("Invalid parenthesis string");
    }
}

```

Complexity Analysis

Time Complexity: $O(3^n)$ the worst case, every '*' can be replaced with '(', ')' or an empty string. For each '*', we have 3 choices, so with k '*' characters, we make $3k$ recursive calls. If the input string has length n, and all are '*', the time complexity becomes exponential.

Space Complexity: $O(n)$, This is due to the maximum depth of the recursive call stack. At most, there are n recursive calls at any time (one for each character).

Optimal Approach

Algorithm

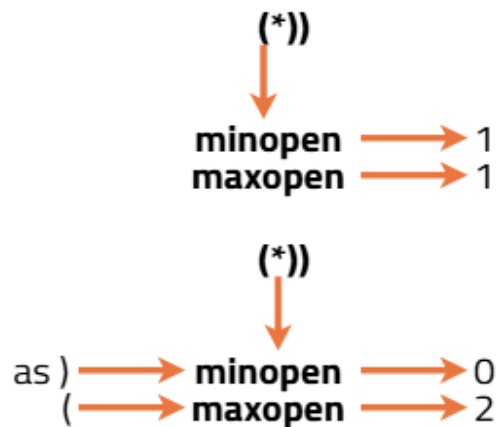
The string contains '(', ')' and '*'. Our task is to decide if it can represent a valid sequence of parentheses. The key point is that '*' can be interpreted in three ways:

- As an opening bracket '('
- As a closing bracket ')'
- As an empty string

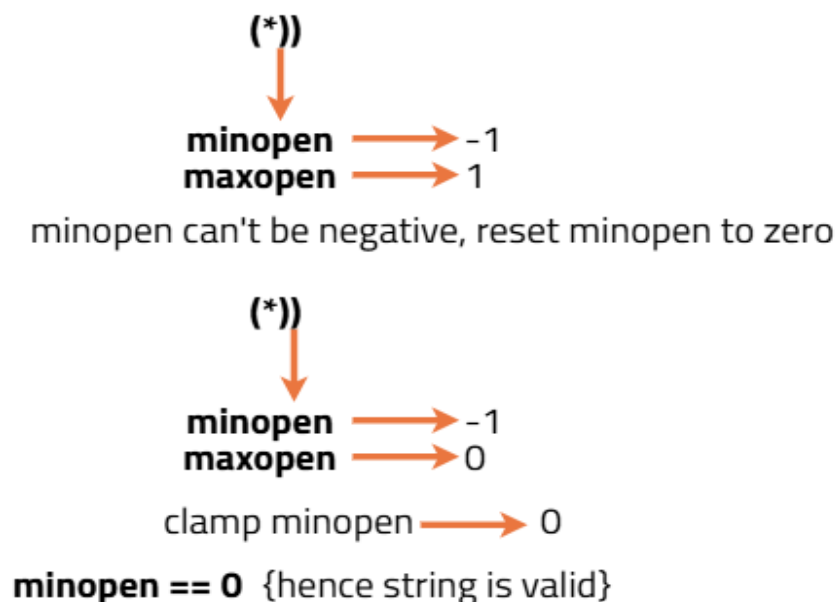
Instead of checking every combination (which is slow), we use a greedy technique that tracks the possible number of unmatched opening brackets:

- We maintain a range [minOpen, maxOpen] to represent the minimum and maximum possible open brackets at each point in the string.
- If we see a '(', both minOpen and maxOpen increase by 1.
- If we see a ')', both minOpen and maxOpen decrease by 1.
- If we see '*', it can behave like '(', ')' or be empty:
 - minOpen decreases by 1 (assuming '*' acts like ')')
 - maxOpen increases by 1 (assuming '*' acts like '(')
- If at any point maxOpen becomes negative, return false — too many closing brackets.
- We also make sure minOpen doesn't drop below 0 — because you can't have negative opening brackets.

If after processing the whole string, minOpen is 0, it means a valid configuration exists.



TUF



Code

```
import java.util.*;
public class Solution {

    // Function to check if the string is a valid parenthesis string using
    greedy approach
    public boolean checkValidString(String s) {

        // Variable to track minimum possible open brackets at current index
        int minOpen = 0;

        // Variable to track maximum possible open brackets at current index
        int maxOpen = 0;

        // Traverse through each character in the string
        for (int i = 0; i < s.length(); i++) {

            // Get current character
```

```

        char c = s.charAt(i);

        // If character is '(', it increases both minOpen and maxOpen
        if (c == '(') {
            minOpen++;
            maxOpen++;
        }

        // If character is ')', it decreases both minOpen and maxOpen
        else if (c == ')') {
            minOpen--;
            maxOpen--;
        }

        // If character is '*', it can be '(', ')' or ''
        else {
            // if '*' is ')'
            minOpen--;
            // if '*' is '('
            maxOpen++;
        }

        // If maxOpen becomes negative, too many closing brackets : invalid
string    if (maxOpen < 0) return false;

        // minOpen can't go below 0, as we can't have negative unmatched '('
        if (minOpen < 0) minOpen = 0;
    }

    // If minOpen is 0 at the end, it's a valid configuration
    return minOpen == 0;
}

// Main class with driver code
class Main {
    public static void main(String[] args) throws java.io.IOException {
        java.util.Scanner sc = new java.util.Scanner(System.in);
        System.out.print("Enter the string: ");
        String input = sc.nextLine();

        Solution sol = new Solution();
        if (sol.checkValidString(input)) {
            System.out.println("Valid parenthesis string");
        } else {
            System.out.println("Invalid parenthesis string");
        }
    }
}

```

Complexity Analysis

Time Complexity: $O(N)$, where N is the length of the input string. We traverse the string once in a single pass, updating min and max possible open brackets at each character.

Space Complexity: $O(1)$, constant space. We use only a few integer variables (minOpen and maxOpen) regardless of the input size.